Load Balancing for a Dynamic Web Application

Let's set up a high availability load balancer for a dynamic Web application written in Java (JSP/Servlet). By 'high availability' we mean that the Web application will be up and running even if a few of the platform components fail or are in maintenance.

To set up a high availability load balancer, we will use two Tomcat 7.0.32 instances behind an Apache 2.14 Web server on a Red Hat Enterprise Linux 6.4 (64-bit) server. This configuration is helpful for any systems administrator who wishes to set up a high performance website based on Java.

This standard configuration consists of three stages:

1. Tomcat AS (Application Server) clustering
2. Tomcat integration to Apache using *mod_jk*
3. Apache configuration

During the entire configuration, we will use the following naming convention: name (and IP) of the Tomcat servers will be tomcatOne (192.168.1.253/24) and tomcatTwo (192.168.1.254/24). The IP on which the Apache server is listening will be 192.168.1.200/24.

Let's start configuring the Tomcat AS cluster, for which it is advisable to instruct Tomcat to stop taking direct requests over the default 8080 port (8443 in case of HTTPS) by commenting the *HTTP(s) Connector* directives in the *server.xml* file as follows:

```
<!-- <Connector address="192.168.1.2XX" port="8080(/8443)"
protocol="HTTP/1.1"
               connectionTimeout="12000"
               redirectPort="8443" /> -->
```

In addition, uncomment the *AJP Connector* directive to allow the Tomcat server to listen over the AJP socket as follows:

```
<Connector address="192.168.1.2XX" port="8009"
protocol="AJP/1.3" redirectPort="8443" />
```

The Apache JServ Protocol (AJP) connector creates a communication link between Apache and Tomcat with the help of the *mod_jk* software module in Apache. Thus, the AJP connector and *mod_jk* handle all the request and response transfers between Apache and Tomcat as illustrated in Figure 1.

Tomcat's AJP connector is written in Java and Apache's *mod_jk* is in C.

Now, let's set up our Tomcat instances to form a cluster by changing the *Engine* and *Cluster* directives for both the Tomcat instances as (shown for tomcatOne, and change the name and IP for tomcatTwo):

```
<Engine name="Catalina" defaultHost="One"
jvmRoute="tomcatOne">
<Cluster className="org.apache.catalina.ha.tcp.
SimpleTcpCluster" channelSendOptions="6">
<Manager className="org.apache.catalina.ha.session.
DeltaManager"
                              expireSessionsOnShutdown="fa
lse"
                              notifyListenersOnReplication
="true" />
```